Python

Amitabha Sanyal

June 2026

Basic Python

Python: Where It Fits

- **Scripting and automation**
  - Python can directly manipulate files and directories (`shutil`), search pathnames (`rglob`), parse text/CSV/JSON (`csv.DictReader`), and access OS services (`os.getcwd`, `os.environ`, `os.system`, `os.makedirs`) through its standard library.
  - It can also orchestrate external programs using `subprocess`.
- **Data analysis and scientific computing**
  - `numpy`, `scipy`, `pandas`, `matplotlib`.
- **Web services and APIs**
  - `Flask`, `Django`, `FastAPI`, `requests`.
- **Machine learning and AI**
  - `scikit-learn`, `PyTorch`, `TensorFlow`.
- **Glue/Wrapper language**
  - Python often provides the control layer over C/C++/Fortran/Rust libraries and tools. Examples: `numpy`, `scipy`, the `gdb-Python API`.

References: Official documentation, Python Tutorial: Brief Tour of the Standard Library, and PyPI: The Python Package Index

Getting Python to Run

- On Linux, use the OS package manager for the system Python:

```
$ sudo apt update
$ sudo apt install python3 python3-pip python3-venv
```

For the newest Python, do not overwrite system Python; install separately or build from source.

- Check the interpreter:

```
$ python3 --version
$ python3 -m pip --version
```

- Run Python in three common ways:

```
$ python3                 # interactive prompt / REPL
$ python3 program.py      # run a script file
$ python3 -m module_name  # run a module as a program
```


References: Using Python on Unix, Command line and environment, Using the Python Interpreter.

Script or Module? The `__main__` Convention

```python
# collatz.py
def collatz(n):
    while n > 1:
        print(n, end=" ")
        n = 3*n + 1 if n % 2 else n // 2
    print(1)
def main():
    n = int(input("Enter n: "))
    collatz(n)
if __name__ == "__main__":
    main()
```

```python
# use_collatz.py

import collatz
collatz.collatz(7)
```

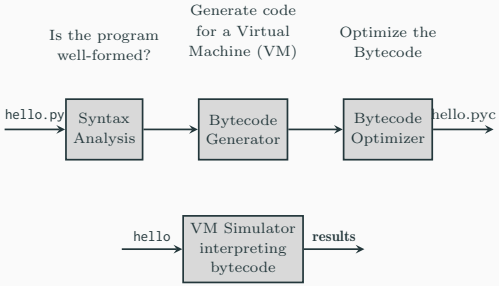
```
# Run the script directly, __name__ = "__main__"
$ python3 collatz.py
7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
```

```
# Import collatz.py as a module, __name__ =
"collatz"
$ python3 use_collatz.py
7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
```

```
# Run in a REPL importing the module, __name__ =
"collatz"
$ python3
>>> import collatz
>>> collatz.main()
...
```

References: __main__: top-level code environment, Modules.

Execution Model: Python is Interpreted



- Python source is first compiled to bytecode.
- The bytecode is then executed by the Python virtual machine.
- Imported modules may have cached bytecode in `__pycache__`.
- Bytecode is an implementation detail of CPython.

References: Execution model, `dis`: bytecode disassembler.

Bytecode

Source:

```
1  import dis
2  ...
3  def collatz(n):
4    while n > 1:
5        print(n, end=' ')
6        if (n % 2):
7            # n is odd
8            n = 3*n + 1
9        else:
10           # n is even
11           n = n//2
12   print(1)
13 def main():
14   dis.dis(collatz)
```

Bytecode: The shaded lines correspond to `n = 3*n + 1`.

```
1   7        20 LOAD_FAST                0 (n)
2            22 LOAD_CONST               4 (2)
3            24 BINARY_MODULO
4            26 POP_JUMP_IF_FALSE       20 (to 40)
5            28 LOAD_CONST               5 (3)
6            30 LOAD_FAST                0 (n)
7            32 BINARY_MULTIPLY
8            34 LOAD_CONST               1 (1)
9            36 BINARY_ADD
10           38 JUMP_FORWARD             3 (to 46)
11      >>   40 LOAD_FAST                0 (n)
12           42 LOAD_CONST               4 (2)
13           44 BINARY_FLOOR_DIVIDE
14      >>   46 STORE_FAST               0 (n)
```

Reference: dis module documentation.

Python Values and Variables

- **C/C++ variable:** denotes a typed storage location.
- **Python variable:** may denote runtime values of different types during execution.
- Therefore, its value cannot, in general, be stored directly in a fixed-size, fixed-type stack location.
- Instead, a Python variable typically stores in stack a reference to a heap-allocated object representing the value.
- The referenced object has its own runtime **identity**, **type**, and **value**.

```
>>> a = [10, 20]
>>> b = a
>>> a is b       # Reference equality
True
>>> a == b       # Value equality
True
```

```
>>> c = [10, 20]
>>> a is c       # Reference equality
False
False
>>> a == c       # Value equality
True
```

References: Data model: Objects, values and types, Execution model: Naming and binding.

Dynamic Type Checking: Consequences

```
1  def fact(n):
2      if (n < 0):
3          return "The argument cannot be negative"
4      elif (n == 0):
5          return 1
6      else:
7          return (n*fact(n-1))
8
9  arg=int(input("arg= "))
10 print(4+fact(arg))
```

- Python does not assign a fixed type to the function `fact`.
- `fact(-1)` returns a string.
- `4 + fact(-1)` fails only when evaluated.
- Type checking is part of the `BINARY_ADD` and `BINARY_MULTIPLY` instructions.
- This is powerful, but errors can hide on untested paths.

References: Data model: objects, values and types, Errors and exceptions, The CPython virtual machine, Python operators.

Python lists: General-purpose sequences

- Lists are ordered.

```
>>> [1,2,3,4] == [4,1,3,2]
False
```

- Lists are heterogeneous (the same list can have different types of objects).

```
>>> a = [21.42, 'foobar', 3, 4, 'bark', False, 3.14159]
>>> mixed = [int, collatz, sin, fact]
>>> empty = []
```

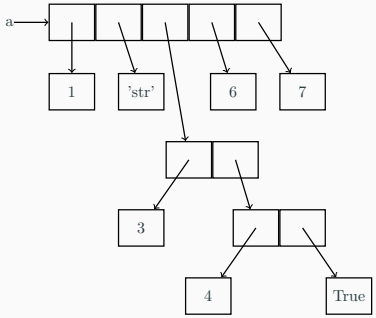
- List elements can be accessed by indices: (0 to 6 or -7 to -1 in the example below)

```
>>> a = [21.42, 'foobar', 3, 4, 'bark', False, 3.14159]
>>> a[-5]
3
>>> a[2:4]              # Slice: from a[2] up to, but not including, a[4]
[3, 4]
```

Python Lists—Nesting and Representation

- Lists can be nested to arbitrary depth.

```
>>> a = [1, 'str', [3, [4, True]], 6, 7]
```



- Lists are represented as dynamic arrays with the varying allocation sizes.
- This tells you how to find the size of any data object (including lists) in Python.
  - Find the size of `a` in the example above, and explain the observed size.

  Here is another site that gives you the internal implementation of lists in Python.

Python lists—Slicing

- List slicing in Python:
  - `a[:n]` is the same as `a[0:n]`
  - `a[n:]` is the same as `a[n:len(a)]`
  - Thus `a[:n] + a[n:] = a = a[:]` (`+` is append)
  - You can also add a stride (step) in a slice.

```
>>> a = ['foo', 'bar', 'baz', 'bark', 'qux', 'cor']
>>> a[0:5:2]
['foo', 'baz', 'qux']
>>> a[5:0]
[]
>>> a[5:0:-1]
['cor', 'qux', 'bark', 'baz', 'bar']
```

List updates and mutability

Each example starts with:

```
>>> a = [1, 'str', [3, [4, True]], 6, 7]
>>> b = a          # b names the same list object
```

append: add one object

```
>>> a.append([8, 9])
>>> a
[1, 'str', [3, [4, True]], 6, 7, [8, 9]]
>>> a is b  # True
```

+=: in-place concatenation

```
>>> a += [8, 9]
>>> a
[1, 'str', [3, [4, True]], 6, 7, 8, 9]
>>> a is b  # True
```

insert: insert object at index

```
>>> a.insert(2, 'name')
>>> a
[1, 'str', 'name', [3, [4, True]], 6, 7]
>>> a is b  # True
```

extend: add elements

```
>>> a.extend([8, 9])
>>> a
[1, 'str', [3, [4, True]], 6, 7, 8, 9]
>>> a is b  # True
```

+: fresh list and reassignment

```
>>> a = a + [8, 9]
>>> a
[1, 'str', [3, [4, True]], 6, 7, 8, 9]
>>> a is b  # False
```

List Comprehension

Python has a powerful feature called list comprehension.

List Comprehension Examples:

```
>>> [x*x for x in [1,2,3,4,5,6,7] if x % 2 == 0]
[4, 16, 36]
```


```
>>> [x+y for x in [1,2,3] for y in [5,6,7]]
[6, 7, 8, 7, 8, 9, 8, 9, 10]
```

- A list comprehension can have any number of generators and guards.
- The variables in a guard should either be from the generators preceding it or from the variables in scope outside the list comprehension.

For Loop Equivalents:

```
result = []
for x in [1,2,3,4,5,6,7]:
    if x % 2 == 0:
    result.append(x*x)
print(result)  # Output: [4, 16, 36]
```

```
result = []
for x in [1,2,3]:
    for y in [5,6,7]:
        result.append(x+y)
print(result)  # Output:
[6,7,8,7,8,9,8,9,10]
```

List Comprehension

Here is quicksort using list comprehension

```
def qsort(lst):
    if lst == []:
        return []
    pivot = lst[0]
    l = qsort([x for x in lst[1:] if x < pivot])
    u = qsort([x for x in lst[1:] if x >= pivot])
    return l + [pivot] + u
```

Manual Memory Management in C and C++

```c
void process(void) {
    int *p = malloc(100 * sizeof(int));
    if (p == NULL)
        return;
    if (!read_data(p))
        return;          // memory leak
    ...
    use_data(p);
    ...
    free(p);
    ...
    use_data(q);         // q and p are aliased
                         // use after free
    ...
    free(q);             // double free
}
```

- Every allocation creates an obligation: the memory must be released **exactly once**.
- It must be released on **every execution path**, including errors and early returns.
- Common failures:
  - **memory leak**: memory never released
  - **dangling pointer**: memory used after release
  - **double free**: memory released twice
- Aliasing makes ownership harder: Which pointer is responsible for freeing the object?
- Modern C++ reduces these risks through RAII and smart pointers; Python manages object lifetime automatically.

Heap Allocation and Garbage Collection

- In Python, objects are allocated dynamically on the heap.
- Names, list slots, dictionary entries, object fields, etc. hold references to objects in the heap.
- When an object becomes unreachable, Python can reclaim it automatically.
- CPython mainly uses reference counting, plus a cyclic garbage collector.

```
def f():
    x = [1, 2, 3]
    y = [x, x]
    return len(y)

f()
# After f returns, x and y are no longer reachable. The garbage collector will eventually reclaim the memory.
```

- Contrast with C/C++: forgetting `free` / `delete` can leak memory.

References: gc: Garbage Collector interface, Data model: Objects, values and types.

Dictionaries (aka maps, hash-tables)

- **Stores objects identified by keys.**

```
>>> phonebook = {"bob": 7387, "alice": 3719, "jack": 7052}
```

- **Dictionary comprehensions work like list comprehensions.**

```
>>> phonebook = {name: number for name, number  in zip(["bob", "alice", "jack"],[7387, 3719, 7052])}
```

- **Access:** Use a syntax similar to lists/arrays:

```
phonebook["bob"]
```

- The key object should be hashable.

References: Dictionaries.

Dictionary operations

Common dictionary operations:

- `d.clear()` – Clears the dictionary `d`.
- `d.get(<key>[, <default>])` – Returns the value associated with `key`. If the key is not present in the dictionary, it returns `default`.
- `d.items()` – Returns a view of key: value pairs in a dictionary. Views are iterable.
- `d.keys()` – Returns a view of keys in a dictionary.
- `d.values()` – Similar.
- `d.pop(<key>[, <default>])` – Removes a key from a dictionary, if it is present, and returns its value.
- `d.update(<obj>)` – Merges a dictionary with another dictionary/list.

References: Dictionaries.

Other Data Structures

- **Tuples:** Immutable containers.

```
>>> point_3d = (4.6, 5.7, -2.1)
```

- **array.array:** A C-like array with all elements having the same type.

```
>>> import array
>>> arr = array.array("f", (1.0, 1.5, 2.0, 2.5))
```

  - These are mutable, dynamic, and homogeneous.
  - They are more time and space efficient. Why?

- **Sets:** Unordered collections.

```
>>> vowels = {"a", "e", "i", "o", "u"}
```

  - There is also a variant called a `multiset`.

References: Python data structures. Complexity of operations on Python data structures.

Functions

Functions are **first-class values**: they can be assigned, passed, returned, and stored just like other values.

A function definition

```
def f(x, y):
    return x**2 + y
```

Assign a function to a variable

```
>>> g = f
>>> g(3, 2)
```

Create an unnamed function

```
>>> (lambda x, y: x**2 + y)(3, 2)
```

A Python `lambda` *is restricted to one expression.*

Pass a function as an argument

```
def square(x):
    return x**2
>>> list(map(square, [1, 2, 3]))
```

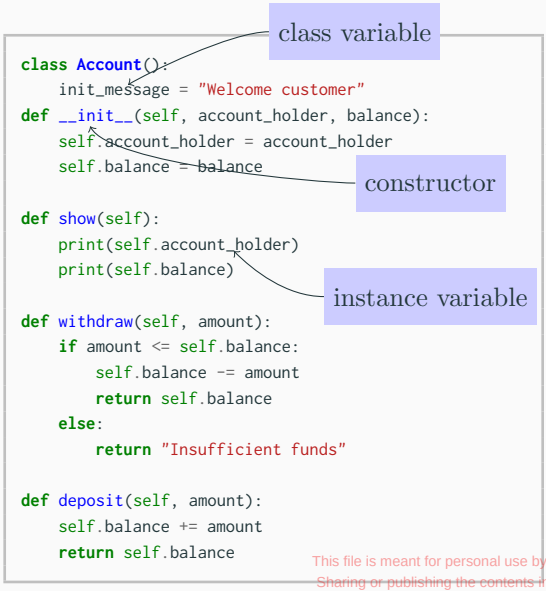
Return a function as a result

```
def make_power(n):
    def power(x):
        return x**n
    return power
>>> square = make_power(2)
>>> square(5)
```

Hold functions in a container

```
def square(x):
    return x**2
def cube(x):
    return x**3
>>> funcs = [square, cube]
>>> [fn(3) for fn in funcs]
```

Classes and Objects




```python
class Account():
    init_message = "Welcome customer"
def __init__(self, account_holder, balance):
    self.account_holder = account_holder
    self.balance = balance

def show(self):
    print(self.account_holder)
    print(self.balance)

def withdraw(self, amount):
    if amount <= self.balance:
        self.balance -= amount
        return self.balance
    else:
        return "Insufficient funds"

def deposit(self, amount):
    self.balance += amount
    return self.balance
```

```
>>> my_account = Account("Amitabha Sanyal", 1000)
>>> my_account.show()
Amitabha Sanyal
1000
>>> print(my_account.init_message)
Welcome customer
```

Classes and Objects: Inheritence

```
class Protected_Account(Account):                    # Derived from Account
"""Modelling a password protected bank account"""
def __init__(self, passwd, account_holder, balance):  # Constructor of derived class
    super().__init__(account_holder, balance)        # Call base constructor
    self.__password = passwd                         # Private member
def withdraw(self, amount, passwd):                  # Modified withdraw method
    if passwd == self.__password:
        return super().withdraw(amount)              # Call base withdraw
    else:
        return "Transaction failed, wrong password"
def deposit(self, amount):                           # Note: show() not defined
    return super().deposit(amount)
```

```
>>> pacc = Protected_Account("lksdj", "Amitabha Sanyal", 1000)
>>> pacc.show()
Amitabha Sanyal
1000
>>> pacc.withdraw(30, "lksd")
'Transaction failed, wrong password'
>>> print(pacc.__password)          # This will raise an AttributeError,
```

Python Classes Are First-Class Objects

- A Python class is itself a runtime object.
- Therefore, a class can be passed to a function, stored in a container, returned as a result, and called to create instances.

```
class Account:
    def __init__(self, holder, balance):
        self.holder = holder
        self.balance = balance

class SavingsAccount(Account):
    interest_rate = 0.04

class CurrentAccount(Account):
    overdraft_limit = 10000
```

```
def open_account(account_class,
                 holder, balance):
    return account_class(holder,
                         balance)

a = open_account(SavingsAccount,
                 "Asha", 5000)

b = open_account(CurrentAccount,
                 "Ravi", 8000)
```

- Here `SavingsAccount` and `CurrentAccount` are passed as values.
- In C++, a class name denotes a **type**, not an ordinary runtime object; the closest analogue is usually a template parameter or a factory function.

Example 1: IPL Matches Scores Problem

The following input represents the scores of players in IPL matches.

```
5
CSKvsRCB:AnujRawat-48,DineshKarthik-38
RCBvsPBKS:ViratKohli-77,ShikharDhawan-45
RCBvsKKR:ViratKohli-83,VenkateshIyer-50
RRvsRCB:ViratKohli-113,JosButtler-100
RCBvsSRH:TravisHead-102,DineshKarthik-83
```

- The first line gives the number of match records.
- Each subsequent line contains match details in the format: `matchName:player-score,player-score,...`

Write a Python program that reads the input, constructs a dictionary mapping each match to its player scores, and produces a sorted list of players with their aggregate runs.

Expected Output:

```
{'CSKvsRCB': {'AnujRawat': 48, 'DineshKarthik': 38}, 'RCBvsPBKS': {'ViratKohli': 77, 'ShikharDhawan': 45},
'RCBvsKKR': {'ViratKohli': 83, 'VenkateshIyer': 50}, 'RRvsRCB': {'ViratKohli': 113, 'JosButtler': 100}, 'RCBvsSRH':
{'TravisHead': 102, 'DineshKarthik': 83}}
[('ViratKohli', 273), ('DineshKarthik', 121), ('TravisHead', 102), ('JosButtler', 100), ('VenkateshIyer', 50),
('AnujRawat', 48), ('ShikharDhawan', 45)]
```

Example 1: Solution

```
num_matches = int(input())
match_dict = {}                              # The outer dictionary
aggregate = {}                               # The aggregate list
for _ in range(num_matches):                 # Each iteration processes a match
    line = input().strip()                   # read a line from input, stripping leading and trailing whitespaces
    match_name, scores_str = line.split(':') # Split the line into match name and scores
    scores_list = scores_str.split(',')      # Split the scores into a list
    match_scores = {}                        # The inner dictionary for a match
    for score in scores_list:
        # Each score is in the form "player-run"
        player, run_str = score.split('-')
        run = int(run_str)
        match_scores[player] = run
        aggregate[player] = aggregate.get(player, 0) + run
    match_dict[match_name] = match_scores
# "sorted" returns a list of aggregated scores, sorted in descending order by runs
sorted_aggregate = sorted(aggregate.items(), key=lambda item: item[1], reverse=True)
# Display the outputs
print(match_dict)
print(sorted_aggregate)
```

Example 2

You are the head of an intelligence unit and you suspect one of your staff to be a spy. The only clue that you have is the suspect's diary, called `MyDiary.txt`, which contains, amongst a rambling text, some email addresses and phone numbers — including your own. Your hypothesis is that the suspect is a spy if he has more conversations with some contact other than yourself.

Write a Python program to:

- First, print your own occurrence frequency on a line in the following format:

```
my frequency: <frequency>
```

- Report all the frequent contacts in the format:

```
Spy alert <spy's contact> <frequency>
Spy alert ...
```

- Otherwise, conclude with:

```
All's well, no spy!!!
```

Example 2

Your program will be invoked like:

```
python3 spy.py <path-to-diary> <mycontact>
# mycontact could be an email or phone no.
```

Sample output:

```
my frequency: 7
Spy alert! cockroach@badguys.com 10
```

Example 2: Regular Expressions

- Here is a semi-formal description of an email id:
  <*email id*> = <*local part*> `@` <*domain*>
  <*local part*> = one or more <*alphanumeric*>s separated by <*dot_or_us*>
  <*alphanumeric*> = one or more letters `[a-zA-Z]` or digits `[0-9]`
  <*dot_or_us*> = the character `.` or the character `_`
  <*domain*> = one or more <*alphanumeric*>s separated by <*dot*>. The last group of <*alphanumeric*>s should consist of <*alphabetic*>s and should have at least two characters
  <*alphabetic*> = one or more letters.
  <*phone_no* > = Ten consecutive digits not starting with 0.
- **Sample email id:** `fxps_ho.4@anhthu.org`
- **Note:** Email and Number will be at a word boundary but may be surrounded or adjacent to punctuations.
- You have to use regular expressions to find the email IDs and phone numbers.

Example 2: Solution

- First import some libraries:

```
import sys
import re
import string
from collections import Counter
```

- Give names to regular expressions for email addresses and phone numbers. Note that `re_email` contains three grouped subexpressions: one capturing group and two non-capturing groups. `re_number` has one.

```
local_part = "(?:[A-Za-z0-9._%+-]+)"
domain     = r"(?:[A-Za-z0-9.-]+\.[A-Za-z]{2,})"
re_email   = r"\b(" + local_part + "@" + domain + r")\b"

re_number  = r'(\b [1-9][0-9]{9}\b )'         # \b...\b for matching at word boundaries only
```

Example 2.

- A `findall` based on `re_email` will return a list of triples, whereas a `findall` based on numbers will result in just a list of numbers.

```
>>> re.findall(re_email, 'as@cse.iitb.ac.in, amit23358@gmail.com')
['as@cse.iitb.ac.in', 'amit23358@gmail.com']
>>> re.findall(re_number, '1234567890, 9999999999')
['1234567890', '9999999999']
```

- We use list comprehension to extract the first element (the match of the outer grouping):

```
emails = []; numbers = []
fp = open(sys.argv[1], "r")
for line in fp.readlines():                    # For each line in MyDiary.txt do
    emails += re.findall(re_email, line)       # Accumulate all emails on the line
    numbers += re.findall(re_number, line)     # Same for phone numbers.
```

Example 2.

- Here is an outline of the rest of the code:

```
count_emails = Counter(emails)      # Creates a dict {email: count}
count_numbers = Counter(numbers)
isNumber = sys.argv[2].isdigit()    # Is my contact phone or email?
spy_dict = {}                       # Initialize a dictionary of spies
isSpy = False                       # To track whether a spy has been found

if isNumber:
    print('my frequency:', count_numbers[sys.argv[2]])
    for no in list(set(numbers)):   # Makes the numbers unique
    if count_numbers[no] > count_numbers[sys.argv[2]]:
        isSpy = True
        spy_dict[no] = count_numbers[no]
else:
...                     # Other actions
# Do the same for emails
```

- Now it is a simple matter to write the rest of the program.

Exceptions

- **Exception Handling:**
  - Errors occur for various reasons: invalid arithmetic, malformed user input, missing files, invalid indices or keys, and failed or timed-out network connections.
  - Exception handling makes programs more robust by detecting errors and responding meaningfully, rather than allowing the program to fail unexpectedly.

Syntax:

```
try:
# Code that might raise an exception
except SomeException as e:
# Code to execute if that exception occurs
```

Example:

```
try:
    int("abc")
except ValueError as e:
    print(e)

invalid literal for int() with base 10: 'abc'
```

Repeated retrials after recovery from:

```
import time

for attempt in range(3):
    try:
        result = contact_server()
        break
    except TimeoutError:
        print("Server did not respond; retrying...")
        time.sleep(1)
else:
    print("Server unavailable after three attempts.")
```

Exception: Examples

- except, else and finally clauses:

```python
try:
    # Code that might raise multiple types of errors
except FileNotFoundError:
    print("File not found!")
except ValueError:
    print("Invalid value!")
```

```python
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Division by zero!")
else:
    print("Result is", result)
finally:
    # This block is always executed
    # Insert cleanup code here
    pass
```

```python
def bar():
    print("bar: starting")
    1 / 0
def foo():
    try:
        print("foo: entering")
        bar()
        print("foo: after bar")       # never executed
    finally:
        print("foo: cleanup in finally")
try:
    foo()
except ZeroDivisionError:
    print("outer level: exception caught")
finally:
    print("Cleanup actions executed")
```

Exceptions and Flow of Control

- When an error occurs in the `try` block, Python immediately stops executing the remaining code in that block.
- The interpreter then searches for a matching `except` clause to handle the raised exception.
- This process involves **stack unwinding**: Python pops stack frames (i.e., exits from functions) until it finds an appropriate exception handler.
- During stack unwinding, any `finally` blocks and context manager clean-up code are executed to release resources.
- If no matching `except` block is found, the exception propagates to the top level, and the program terminates with a traceback.

Predefined Exceptions in Python

- **Exception:** Base class for all built-in exceptions.
- **IndexError:** Raised when a sequence index is out of range.
- **KeyError:** Raised when a dictionary key is not found.
- **TypeError:** Raised when an operation is applied to an inappropriate type.
- **ValueError:** Raised when an argument has the right type but an inappropriate value.
- **ZeroDivisionError:** Raised when division or modulo by zero occurs.
- **FileNotFoundError:** Raised when a file or directory is requested but does not exist.
- **ImportError:** Raised when an import statement fails to load a module.

For a complete list, visit: https://docs.python.org/3/library/exceptions.html

Modules and Imports

- A **module** is a `.py` file containing definitions and statements.
- Importing a module gives access to the names defined in it:
  - function names, class names, variable names, imported module names, etc.
  - these names live in the module's own namespace.

```python
# stats.py
version = "1.0"
def mean(xs):
    return sum(xs) / len(xs)
```

```python
# use_stats.py
import stats

marks = [8, 9, 10]
print(stats.mean(marks))
print(stats.version)
```

```python
# use_stats.py
from numpy import *   # Also defines a mean
from stats import *
marks = [8, 9, 10]
print(mean(marks))
print(version)
```

- Prefer `import stats`: Keeps the origins of the names visible.
- Avoid `from modulename import *`: it imports an unknown set of names.

Inspecting Namespaces

Example module

```
# inspect_demo.py
import math
x = 10
def f(a):
    local = a + 1
    return locals()
def show():
    return globals()
class C:
    y = 20
    def method(self):
        print(locals())
        return self.z + C.y
obj = C()
obj.z = 30
```

Try these inspections

```
>>> inspect_demo.show()["x"]
10
>>> "math" in inspect_demo.show()
True
>>> inspect_demo.f(5)
{'a': 5, 'local': 6}
>>> inspect_demo.C.__dict__["y"]
20
>>> inspect_demo.C.__dict__
... 'y': 20, 'method': <function C.method at 0x7278488f6710>...
>>> vars(inspect_demo.obj)
{'z': 30}
>>> "method" in dir(inspect_demo.obj)
True
```

- `globals()` and `locals()` inspect namespaces.
- `__dict__` and `vars()` expose attribute mappings.
- `dir()` lists available names; it does not return their values.

Packages, Standard Library, and PyPI

- A **package** organizes related modules using dotted names.

```
project/
    analysis/
        __init__.py
        io.py
        stats.py
        main.py
```

```
from analysis import stats
from analysis.io import read_marks
```

- The **standard library** is shipped with Python: `sys`, `os`, `pathlib`, `re`, `collections`, `json`, `argparse`, ... Third-party packages are commonly installed from **PyPI**.
- A regular package is a directory with `__init__.py`; importing the package executes this file and initializes the package namespace.

References: Packages, The Python Standard Library, Packaging Python Projects, PyPI.

Reading and Writing Files

- Use `with open(...)` so files are closed automatically.
- Text mode returns strings; binary mode returns bytes.

```
# Text file
with open("marks.txt", "r", encoding="utf-8") as f:
    for line in f:
        roll, marks = line.split()
        print(roll, int(marks))

# Binary file
with open("image.png", "rb") as f:
    header = f.read(8)
```

- Common modes: `"r"`, `"w"`, `"a"`, `"rb"`, `"wb"`.

References: Reading and Writing Files, open.

Paths with `pathlib`

- Prefer `pathlib.Path` over manual string manipulation.
- Path operations become object operations.

```python
from pathlib import Path

data_dir = Path("data")
input_file = data_dir / "marks.txt"
output_file = data_dir / "summary.txt"

print(input_file.name)      # marks.txt
print(input_file.suffix)    # .txt
print(input_file.exists())  # False

with input_file.open("r", encoding="utf-8") as f:
    lines = f.readlines()

output_file.write_text("Number of lines: " + str(len(lines)) + "\n",
encoding="utf-8")
```

References: pathlib: Object-oriented filesystem paths, File and Directory Access.

Command-line Arguments with `sys.argv`

- Small scripts often receive filenames from the command line.
- `sys.argv` is the list of command-line arguments.
- `sys.argv[0]` is the script name; later entries are user arguments.

```python
# count_lines.py
from pathlib import Path
import sys

if len(sys.argv) != 2:
    print("Usage: python3 count_lines.py FILE")
    sys.exit(1)
path = Path(sys.argv[1])

with path.open("r", encoding="utf-8") as f:
    nlines = sum(1 for line in f)

print(path, "has", nlines, "lines")
```

References: sys.argv, argparse: Parser for command-line options.

Debugging

- Use `assert` for conditions that must be true.
- Use `breakpoint()` to enter the debugger at a chosen point, or do a postmortem debugging.

```python
import pdb
def divide(total, count):
    assert(count > 0), "Count must be positive"
    return total / count
def average(scores):
    return divide(sum(scores), len(scores))
def parse_record(line):
    name, raw = line.split(":")
   scores = [int(x) for x in raw.split(",") if x]
    return name, average(scores)
def main():
    lines = ["Amit:80,90", "Vikram:"]
    for line in lines:
        name, avg = parse_record(line)
        print(name, avg)
main()
```

```
python3 -m pdb program.py

args            function arguments and their values
p expr          evaluate and print an expression
list            show nearby source lines
where           show the call stack
up/down         select another stack frame
next            next line, step over function calls
step            next line, enter function calls
break 20        set a breakpoint at line 20
break 20, x==0  conditional breakpoint
continue        resume execution
quit            terminate debugging
help            list commands
locals()        local variables and args
display x
```

- Good debugging recipe: reproduce, minimize, inspect, fix, retest.

Virtual Environments and `pip`

- A virtual environment keeps project packages separate from system Python.
- Avoid installing project dependencies globally.

```
$ python3 -m venv .venv
$ source .venv/bin/activate
$ (.venv) which python
/home/as/.../.venv/bin/python
$ (.venv) python -m pip install requests pandas
$ (.venv) python -m pip freeze > requirements.txt
$ (.venv) deactivate
$
```

- Use `python -m pip` so `pip` belongs to the selected Python.

References: venv: Creation of virtual environments, Installing Python Modules.